AVALIAÇÃO PRÁTICA – PARTE I

A primeira parte da avaliação prática deverá estar integralmente implementada até o dia **22/10/2026**. O discente deverá comparecer à avaliação com o projeto funcional e pronto para execução. Na data da avaliação, serão divulgados os requisitos e os critérios de avaliação da segunda parte da prova. A composição da nota final seguirá a seguinte distribuição:

- Parte I: peso de 25%
- Parte II: peso de 75%

A prova prática consiste no desenvolvimento de uma aplicação para gerenciamento de clientes, veículos e locações, conforme os requisitos funcionais e não funcionais estabelecidos na especificação do projeto.

A solução deve ser baseada em uma arquitetura em camadas (Figura 01), contemplando a separação entre as responsabilidades de apresentação (Front-end), lógica de negócios e persistência de dados (Back-end), com uso de frameworks apropriados para cada parte.

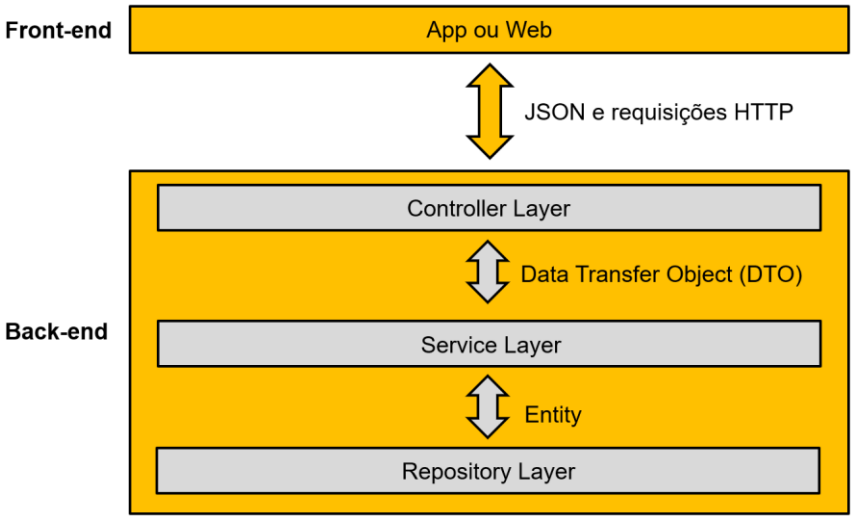


Figura 01: Arquitetura em Camadas

A escolha das tecnologias a serem utilizadas no desenvolvimento do sistema fica a critério do discente, incluindo frameworks, linguagens de programação e Sistemas Gerenciadores de Banco de Dados (SGBDs). A seguir, são apresentados alguns exemplos de tecnologias que podem ser utilizadas:

- Front-end: Angular, React, Vue.js
- Back-end: Spring Boot (Java), Laravel (PHP), Node.js (JavaScript/TypeScript)
- SGBD: MySQL, PostgreSQL, MongoDB

Observação: Independentemente das tecnologias escolhidas, o discente deverá garantir que sejam adequadas aos requisitos da avaliação e permitam a implementação de todas as funcionalidades exigidas.

Projeto da Aplicação Back-end

O desenvolvimento da aplicação back-end deve atender aos requisitos funcionais e não funcionais descritos a seguir:

- **Modelo de Dados**
  O modelo de dados é composto por três classes principais: **Usuario**, **Veiculo** e **Locacao**, além das enumerações **Perfil e Situacao**. O discente deverá seguir rigorosamente o modelo fornecido,

respeitando as classes, os atributos, os tipos de dados, os relacionamentos e os valores das enumerações. Não será permitida a inclusão, remoção ou alteração de classes, atributos ou valores das enumerações do modelo de dados. A implementação de métodos, construtores e anotações necessárias ao funcionamento da aplicação é permitida, desde que a estrutura do modelo seja preservada.

```
public class Usuario {
    private Long id;
    private String login;
    private String senha;
    private Perfil perfil;
}
```

```
public class Veiculo {
    private Long id;
    private String placa;
    private String descricao;
    private Boolean disponivel;
}
```

```
public class Locacao {
    private Long id;
    private Veiculo veiculo;
    private Situacao situacao;
    private Usuario cliente;
    private Usuario funcionario;
}
```

```
public enum Perfil {
    CLIENTE, FUNCIONARIO
}
```

```
public enum Situacao {
    EM_ANDAMENTO, FINALIZADA
}
```

Requisitos de Modelagem

- Usuário
  - Todos os campos são obrigatórios.
  - O campo login deve ser único.
  - O perfil deve ser armazenado utilizando EnumType.STRING.
- Veículo
  - Todos os campos são obrigatórios.
  - O campo placa deve ser único.
  - O campo disponivel é controlado pela aplicação e deve iniciar com valor true.
- Locação
  - Todos os campos são obrigatórios.
  - Cada locação deve estar associada a um veículo, a um cliente e ao funcionário responsável, por meio de relacionamentos muitos-para-um.
  - A situação deve ser armazenada utilizando EnumType.STRING e iniciar como EM_ANDAMENTO.

Data Transfer Object (DTO)

O padrão DTO é utilizado para transportar dados entre as camadas da aplicação. Sua principal função é encapsular os dados que realmente precisam ser expostos, evitando o envio de informações desnecessárias ou sensíveis. Crie DTOs de entrada (RequestDTO) e saída (ResponseDTO) com as seguintes características:

DTO	Campos
UsuarioRequestDTO	login, senha, perfil
UsuarioResponseDTO	id, login, perfil
UsuarioSenhaRequestDTO	senha
VeiculoRequestDTO	placa, descricao
VeiculoResponseDTO	id, placa, descricao, disponivel
LocacaoRequestDTO	veiculoId, clienteId
LocacaoResponseDTO	id, veiculoId, situacao, clienteId, funcionarioId
AutenticacaoRequestDTO	login, senha
AutenticacaoResponseDTO	token

- Observações:
  - O identificador funcionarioId não deve ser informado no DTO de entrada LocacaoRequestDTO.

- Na criação de uma locação, o funcionário responsável deve ser identificado a partir do token JWT do usuário autenticado.

Arquitetura da Aplicação

A aplicação deverá seguir a arquitetura apresentada na Figura 01, organizada em camadas para garantir a separação de responsabilidades, a organização e a manutenção do código. Cada camada deverá desempenhar uma função específica:

- **Repositório (Repository**): componentes responsáveis pelo acesso e pela persistência dos dados, utilizando os mecanismos disponibilizados pela tecnologia escolhida.
- **Serviço (Service)**: classes responsáveis pela implementação das regras de negócio e validações do sistema.
- **Controle (Controller)**: classes responsáveis por disponibilizar os endpoints da API e processar as requisições HTTP.
- **Entidade (Entity)**: classes que representam o modelo de dados e seus relacionamentos, utilizadas na persistência das informações.
- **DTO (Data Transfer Object)**: classes utilizadas na entrada e saída de dados da API, evitando a exposição direta das entidades.
- **Segurança (Security)**: componentes responsáveis pela autenticação utilizando JWT e pela autorização de acesso conforme o perfil do usuário.

Regras e restrições

Implemente as funcionalidades necessárias no back-end e no front-end, garantindo o cumprimento das seguintes regras de negócio. As validações e restrições deverão ser aplicadas obrigatoriamente no back-end, enquanto o front-end deverá oferecer as funcionalidades e apresentar as mensagens correspondentes ao usuário.

- **R01 – Cadastro de usuário**: não deve ser permitido cadastrar dois usuários com o mesmo login.
- **R02 – Cadastro e alteração de veículo**: não deve ser permitido cadastrar dois veículos com a mesma placa. Na alteração, apenas a descrição poderá ser modificada pelo usuário.
- **R03 – Disponibilidade inicial**: todo novo veículo deve iniciar com disponivel = true.
- **R04 – Criação da locação**: somente um FUNCIONARIO pode criar uma locação. O funcionário responsável deve ser identificado pelo JWT. O usuário informado como cliente deve possuir o perfil CLIENTE.
- **R05 – Veículo da locação**: uma locação somente pode ser criada para um veículo disponível. Ao iniciar a locação, o veículo deve passar para disponivel = false.
- **R06 – Situação da locação**: toda nova locação deve iniciar com Situacao = EM_ANDAMENTO.
- **R07 – Consulta de locações**: o CLIENTE poderá consultar apenas as locações das quais participa como cliente. O FUNCIONARIO poderá consultar apenas as locações pelas quais é responsável.
- **R08 – Finalização da locação**: somente locações EM_ANDAMENTO podem ser finalizadas. O FUNCIONARIO poderá finalizar apenas as locações pelas quais é responsável. Ao finalizar, a situação passa para FINALIZADA e o veículo volta para disponivel = true.
- **R09 – Exclusão de veículo**: não deve ser permitida a exclusão de veículos associados a locações, independentemente da situação dessas locações.

Endpoints

A API deve disponibilizar os seguintes endpoints para cada entidade:

Método	URL	Descrição
POST	http://localhost:8080/usuarios	Cadastrar usuário
GET	http://localhost:8080/usuarios/existe?login={login}	Verificar existência do login
POST	http://localhost:8080/login	Autenticar usuário
GET	http://localhost:8080/usuarios/meus-dados	Consultar os próprios dados. ID obtido do JWT
PATCH	http://localhost:8080/usuarios/senha	Alterar a própria senha. ID obtido do JWT
GET	http://localhost:8080/usuarios/clientes	Listar todos os clientes

Método	URL	Descrição
GET	http://localhost:8080/veiculos	Listar todos os veículos
GET	http://localhost:8080/veiculos/{id}	Consultar veículo por ID
GET	http://localhost:8080/veiculos/existe?placa={placa}	Verificar existência da placa
POST	http://localhost:8080/veiculos	Cadastrar veículo
PATCH	http://localhost:8080/veiculos/{id}	Alterar descrição
DELETE	http://localhost:8080/veiculos/{id}	Excluir veículo

Método	URL	Descrição
POST	http://localhost:8080/locacoes	Criar locação; situação inicia EM_ANDAMENTO
GET	http://localhost:8080/locacoes/minhas	Cliente ou funcionário lista somente suas locações
GET	http://localhost:8080/locacoes/{id}	Consultar locação conforme participação/perfil
PATCH	http://localhost:8080/locacoes/{id}/finalizar	Finalizar locação e liberar o veículo

Configurações de Acesso

- A tabela abaixo define os perfis autorizados para cada *endpoint*. O símbolo 🌐 indica *endpoint* público, ☑ indica acesso permitido e ✗ indica acesso negado.

Método	Endpoint	CLIENTE	FUNC.	Descrição
POST	http://localhost:8080/usuarios	🌐	🌐	Cadastrar usuário
GET	http://localhost:8080/usuarios/existe?login={login}	🌐	🌐	Verificar a existência de usuário com o login informado
POST	http://localhost:8080/login	🌐	🌐	Autenticar usuário
GET	http://localhost:8080/usuarios/meus-dados	☑	☑	Consultar os próprios dados - usuarioId obtido do JWT
PATCH	http://localhost:8080/usuarios/senha	☑	☑	Alterar a própria senha - usuarioId obtido do JWT
GET	http://localhost:8080/usuarios/clientes	✗	☑	Listar todos os clientes
GET	http://localhost:8080/veiculos	✗	☑	Listar todos os veículos
GET	http://localhost:8080/veiculos/{id}	☑	☑	Consultar veículo pelo ID
GET	http://localhost:8080/veiculos/existe?placa={placa}	✗	☑	Verificar a existência de veículo com a placa informada
POST	http://localhost:8080/veiculos	✗	☑	Cadastrar veículo
PATCH	http://localhost:8080/veiculos/{id}	✗	☑	Alterar a descrição do veículo
DELETE	http://localhost:8080/veiculos/{id}	✗	☑	Excluir veículo
POST	http://localhost:8080/locacoes	✗	☑	Criar locação - usuarioId obtido do JWT
GET	http://localhost:8080/locacoes/minhas	☑	☑	Listar as próprias locações - usuarioId e perfil obtido do JWT
GET	http://localhost:8080/locacoes/{id}	☑	☑	Consultar locação- usuarioId e perfil obtido do JWT
PATCH	http://localhost:8080/locacoes/{id}/finalizar	✗	☑	Finalizar locação - usuarioId e perfil obtido do JWT

Projeto da Aplicação Front-end

O desenvolvimento da aplicação front-end deve atender aos requisitos funcionais e não funcionais descritos a seguir. Os protótipos apresentados têm caráter ilustrativo, representando possíveis interfaces do sistema. O discente tem autonomia para definir o design e o layout que considerar mais adequados às necessidades do projeto.

- **Requisito Funcional - Cadastro e Autenticação de Usuário**

  O sistema deve oferecer funcionalidade para o cadastro de novos usuários, exigindo, obrigatoriamente, os seguintes dados: login, senha e perfil (Figura 02). O login deve ser único, não sendo permitido o registro de usuários com login duplicado. O acesso ao sistema deve ser restrito aos usuários previamente cadastrados na base de dados.



Figura 02: Prototipo do cadastro e autenticação de usuários

- **Requisito Funcional - Autenticação de Usuário**

  O acesso ao sistema deve ser realizado por meio da mesma tela de login utilizada pelos usuários. No entanto, o sistema deve identificar automaticamente o tipo de usuário autenticado, garantindo que funcionários tenham acesso às funcionalidades administrativas e clientes apenas aos recursos destinados ao seu perfil.

- **Requisito Funcional - Menu de Acesso**

  Após a autenticação, o sistema deve identificar o perfil do usuário (cliente ou funcionário) e exibir seu login na interface da aplicação, juntamente com as funcionalidades correspondentes ao seu tipo de acesso (Figura 03).

Funcionalidade	Cliente	Funcionário	Descrição
Meus Dados	X	X	Visualização dos próprios dados e alteração da senha
Minhas Locações	X		Consultar locações do cliente
Veículos		X	Cadastrar e gerenciar veículos
Gerenciar Locações		X	Acessar e gerenciar as locações
Sair	X	X	Encerrar a sessão

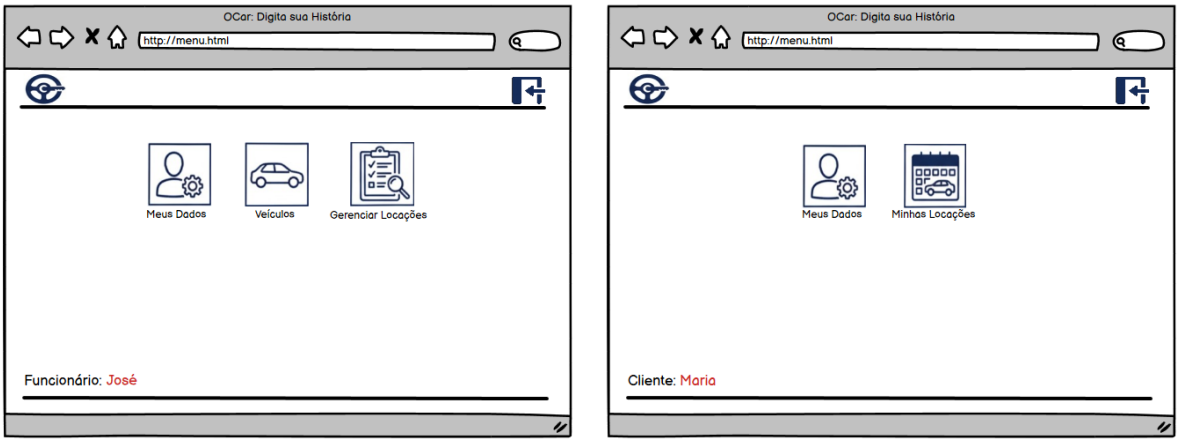


Figura 03: Protótipo menu de acesso

- **Requisito Funcional - Meus Dados**
  O sistema deve permitir que o usuário autenticado visualize seus dados e altere sua senha (Figura 04).

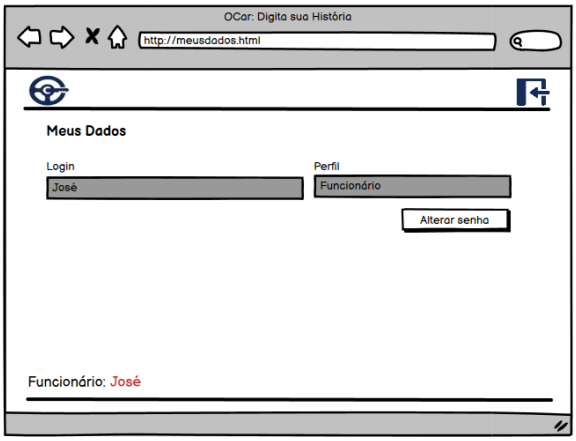


Figura 04: Protótipo Meus Dados

- **Requisito Funcional - Veículos**
  O sistema deve permitir que o funcionário cadastre, consulte, altere e exclua veículos (Figura 05). No cadastro, os campos descrição e placa são obrigatórios, e a disponibilidade deve iniciar como 'SIM', sem possibilidade de alteração manual. Na edição, somente a descrição poderá ser modificada.

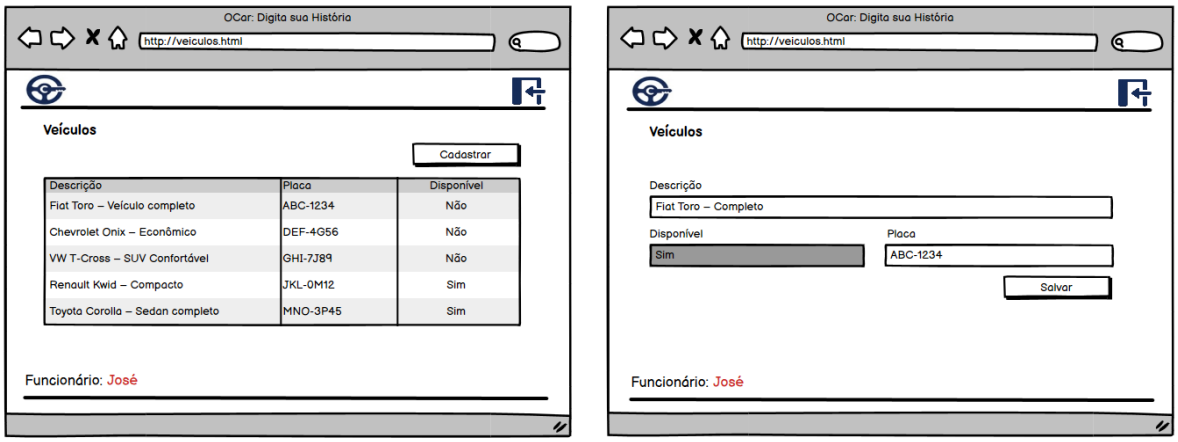


Figura 05: Protótipo Veículos

- **Requisito Funcional - Gerenciar Locações**
  O sistema deve permitir que o funcionário visualize e gerencie as locações (Figura 06), selecionando o cliente e o veículo. Ao cadastrar uma locação, sua situação deve ser definida automaticamente como 'EM ANDAMENTO', e a disponibilidade do veículo deve ser alterada para 'NÃO'. O sistema também deve permitir a finalização de uma locação em andamento, alterando sua situação para 'FINALIZADA' e a disponibilidade do veículo para 'SIM'.

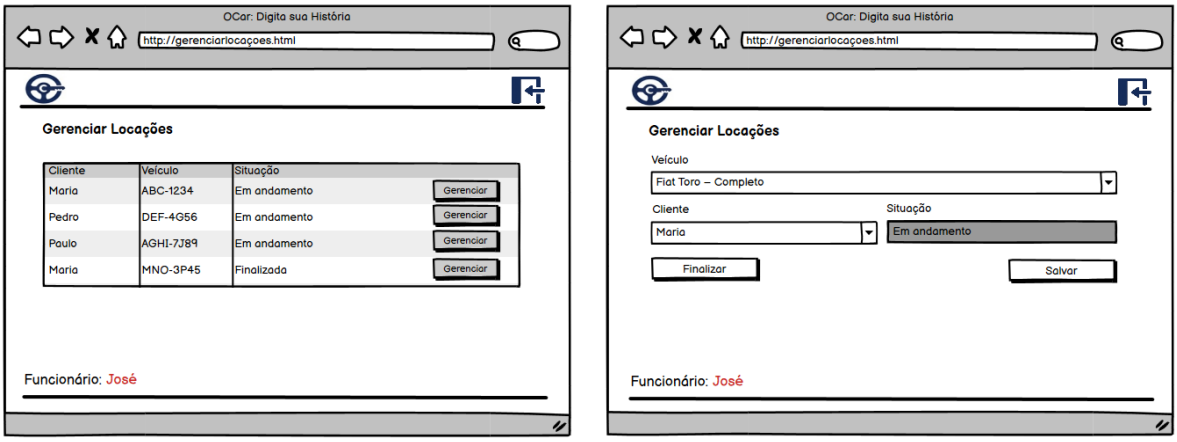


Figura 06: Protótipo Gerenciar Locações

- **Requisito Funcional - Minhas Locações**

  O sistema deve exibir a lista de locações do usuário (cliente) autenticado (Figura 07).

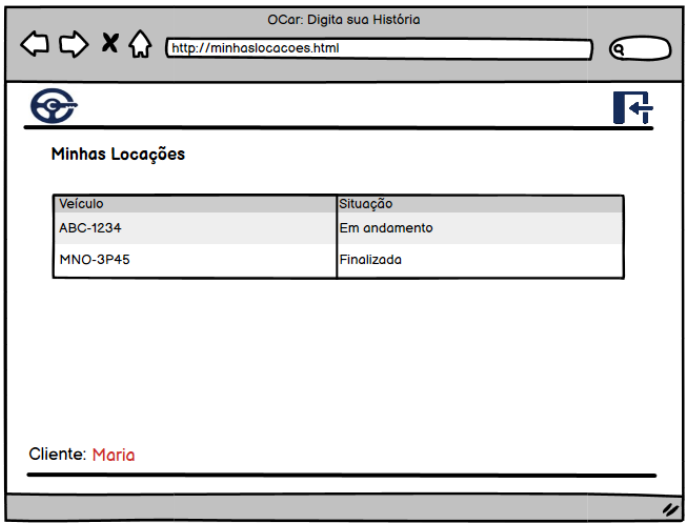


Figura 07: Protótipo Minhas Locações

- **Requisito Não Funcional - Responsividade**

  O sistema deve ser desenvolvido com foco em responsividade, garantindo que a interface se adapte de forma adequada a diferentes tamanhos de tela, como os de computadores, notebooks e smartphones. O discente deve assegurar que todas as funcionalidades estejam acessíveis e visualmente organizadas independentemente da resolução ou tipo de tela utilizada pelo usuário.

Critérios de avaliação

Parte I

- Aplicação Back-end **[12,5 pontos]**
  - Serão avaliados a organização da aplicação em camadas (repositório, serviço e controlador), o funcionamento dos endpoints da API REST, as operações de CRUD e a implementação de autenticação e controle de acesso.
  - A validação dos endpoints será realizada por meio do arquivo do Insomnia disponibilizado pela banca, considerando URLs, métodos HTTP, requisições e autenticação.
- Aplicação Front-end **[12,5 pontos]**
  - Serão avaliados o cadastro e a autenticação de usuários, o gerenciamento de veículos e locações, a integração com o back-end, a atualização dos dados e a usabilidade e responsividade da interface.

Parte II

- Os requisitos e os critérios de avaliação referentes à Parte II serão divulgados no início da avaliação prática que será realizada no dia **23/10/2026**, na sala 47 do Bloco B (Laboratório de Informática I), com duração máxima de 4 horas, entre 13h30 e 17h30.